1️⃣ Next Word Prediction (Local .txt)

📁 nwp.txt

```
deep learning is powerful
deep learning is interesting
deep learning models are useful
```

✅ Code

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load local file
text = open("nwp.txt", encoding="utf-8").read().lower()

tok = Tokenizer()
tok.fit_on_texts([text])
words = tok.texts_to_sequences([text])[0]
vocab = len(tok.word_index) + 1

seqs = [words[:i+1] for i in range(1, len(words))]
max_len = max(len(s) for s in seqs)
seqs = pad_sequences(seqs, maxlen=max_len, padding='pre')

X, y = seqs[:, :-1], seqs[:, -1]

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab, 50, input_length=max_len-1),
    tf.keras.layers.LSTM(100),
    tf.keras.layers.Dense(vocab, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy', optimizer='adam')
model.fit(X, y, epochs=200, verbose=0)

seed = "deep learning is"
seq = pad_sequences([tok.texts_to_sequences([seed])[0]], maxlen=max_len-1,
padding='pre')
pred = np.argmax(model.predict(seq, verbose=0))
```

```
for w, i in tok.word_index.items():
    if i == pred:
        seed += " " + w

print("Predicted:", seed)
```

2️⃣ Sentiment Classification (Local CSV)

📁 **sentiment.csv**

```
text,label
I love NLP,1
This movie is amazing,1
I hate bugs,0
Debugging is boring,0
```

✅ **Code**

```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load local CSV
data = pd.read_csv("sentiment.csv")
texts = data["text"]
labels = data["label"]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)

X = pad_sequences(tokenizer.texts_to_sequences(texts), maxlen=10)
y = np.array(labels)

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(1000, 32, input_length=10),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=10, verbose=0)

loss, acc = model.evaluate(X, y)
print("Accuracy:", acc)
```

3️⃣ Named Entity Recognition (Local Text Dataset)

📁 **ner.txt**

```
Sundar Pichai is CEO of Google
Elon Musk founded SpaceX
I live in India
```

✅ **Code (Using spaCy – no online dataset)**

```
import spacy

nlp = spacy.load("en_core_web_sm")

# Load local file
with open("ner.txt", "r", encoding="utf-8") as f:
    texts = f.readlines()

for text in texts:
    doc = nlp(text.strip())
    print("\nSentence:", text.strip())
    for ent in doc.ents:
        print(ent.text, "->", ent.label_)
```

(Install once: python -m spacy download en_core_web_sm)

4️⃣ Information Retrieval (Local Dataset)

📁 **documents.txt**

```
information retrieval extracts relevant documents
machine learning improves search engines
deep learning is part of artificial intelligence
natural language processing analyzes text
```

✅ **Code**

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Load local documents
with open("documents.txt", "r", encoding="utf-8") as f:
    documents = f.read().splitlines()

model = SentenceTransformer('all-MiniLM-L6-v2')
doc_embeddings = model.encode(documents)

def retrieve(query):
    query_embedding = model.encode([query])
    scores = cosine_similarity(query_embedding, doc_embeddings)
    return documents[np.argmax(scores)]

print("Result:", retrieve("deep learning methods"))
```